Native Mobile And Aws Delivery Plan

NetworkPeer Native Mobile and AWS Delivery Plan

Decision Summary

NetworkPeer already has the correct control plane for native apps: Fastify owns authentication and authorization, PostgreSQL/PostGIS owns lifecycle and financial

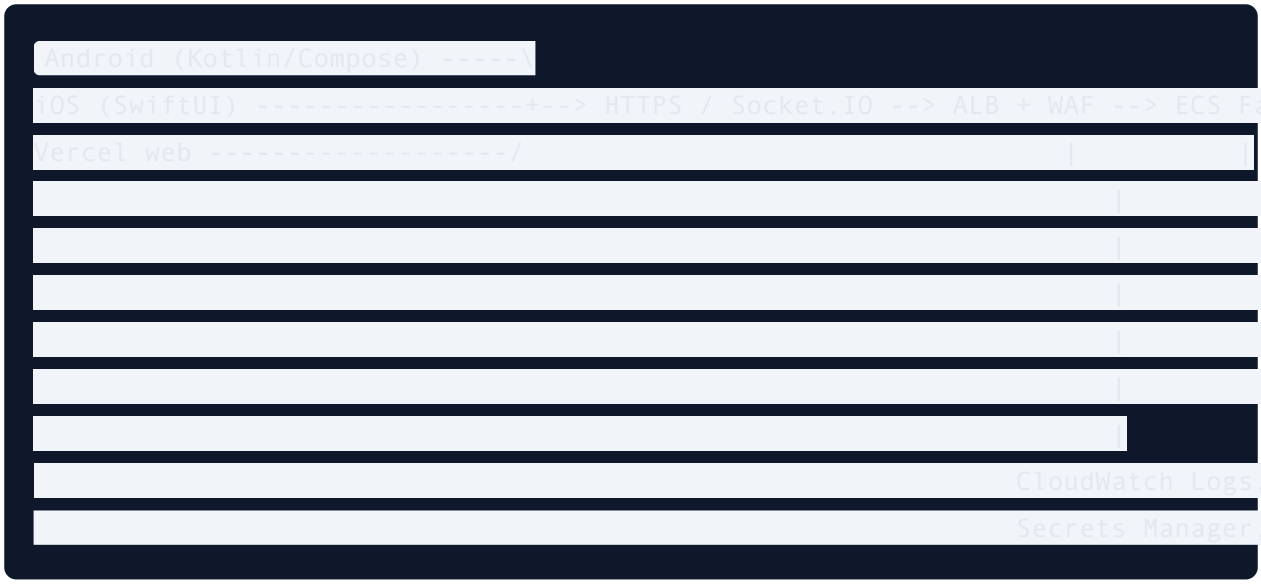
truth, Redis/BullMQ transports durable outbox work, Stripe settles payments, and S3 stores version-pinned evidence. Android and iOS must use that API. They must never connect directly to PostgreSQL, Redis, or S3 with AWS credentials.

This plan keeps the existing API and moves the long-running API and worker to AWS ECS/Fargate. It deliberately does not introduce Cognito, Lambda/API Gateway, AppSync, DynamoDB, or SNS as replacement systems. Those services would duplicate working authentication, realtime, database, queue, or push controls and would increase migration risk before the native launch.

Delivery Status Update

The original phased plan is now implemented in source. The canonical current-state handover is `NETWORKPEER_END_TO_END_DELIVERY_GUIDE.md`, including the AWS release gate, client evidence review, data-only account-validated push delivery, and exact external activation requirements. AWS resources, production payment activity, DNS changes, and store submissions remain intentionally pending authorised account owners.

Target Architecture



The native applications receive a NetworkPeer JWT from the existing OTP flow and use a refresh token stored in Android Keystore/iOS Keychain. The API returns a short-lived presigned S3 POST only after it reserves evidence metadata in PostgreSQL. The mobile application uploads directly to S3 with the opaque POST

fields, then calls the API to verify the immutable object version, checksum, size, and MIME type.

Mobile Data Rule

“Matching the database” means matching the API’s authorised projections and lifecycle enums, not granting a phone access to table structures or database credentials.

Backend truth	Native representation
`users`, `worker_profiles`	Auth session and worker-safe profile projection.
`jobs`, `job_subtasks`	`Job`, `WorkerJobSummary`, `WorkerJobDetail`, and `JobSubtask` API models.
PostGIS locations	GeoJSON `Point` using `[longitude, latitude]`; native location APIs submit latitude/longitude only to `/worker/location`.
`job_subtask_media`	Reservation/target/confirmation flow; S3 key, version ID, and bucket stay server-side.
`wallet_ledger`, payment operations	Wallet summaries and settlement states. Mobile never computes balances.
`sync_events`, notifications	Decimal-string cursor, persisted locally; Socket.IO is a hint and HTTP sync is authoritative.

The initial contract lives in `packages/api-contracts/networkpeer-mobile-contract.yaml`. Backend Zod schemas and route files remain the runtime authority until Phase 1 produces generated OpenAPI clients.

AWS Services

Use in the first deployable environment

Service	Role in NetworkPeer	Why it is essential
IAM Identity Center and IAM roles	Human access, GitHub deployment identity, ECS task identities	Replaces shared access keys with least privilege and auditable role assumption.
AWS Organizations, CloudTrail, Budgets	Account governance, audit trail, cost alarms	Enable before provisioned resources. Use the boss's account policies rather than creating parallel administrator users.
VPC, subnets, security groups, NAT gateway	Private network for API, worker, Postgres, Redis	Keeps data services off the public internet. Use two AZs for production; a single NAT gateway is acceptable only for a cost-conscious demonstration environment.
ECR	Immutable API/worker images	ECS deploys the existing Docker image; image tags use Git SHA.
ECS on Fargate	Long-running Fastify API and separate BullMQ worker	Supports Socket.IO and background processing without rewriting the Node application around Lambda limits.
Application Load Balancer and ACM	Public TLS endpoint and WebSocket upgrades	Required for HTTPS, native/web API access, Stripe webhooks, and Socket.IO.
AWS WAF	Edge rate limiting and common exploit filtering	Complements application/Redis rate limits; it does not replace them.
RDS for PostgreSQL with PostGIS	Authoritative jobs, ledger, lifecycle, and outboxes	Use an engine/version that supports `postgis`; apply existing migrations with a one-shot migration task.
ElastiCache for Redis with TLS	OTPs, refresh rotation, rate limits, BullMQ transport	Existing code requires `rediss://` in hosted

Service	Role in NetworkPeer	Why it is essential
		production. Redis is never financial/workflow truth.
S3	Private, versioned, encrypted evidence storage	Existing S3 verifier requires versioning, encryption, object tags, checksum metadata, and block-public-access.
Secrets Manager	Runtime secrets for API, worker, and migration task	Keeps database URLs, Stripe/Twilio credentials, JWT secrets, and Redis URL out of source, CI logs, and mobile builds.
CloudWatch Logs, metrics, alarms	ECS/RDS/Redis/S3 operational visibility	Retain structured Pino logs, alert on task failures, API 5xx, RDS pressure, and Redis memory.
AWS Backup	RDS snapshot policy and recovery governance	Back up the actual source of truth; rehearse restore before production.
Route 53	DNS for API/domain, if the hosted zone is in AWS	Optional if DNS remains elsewhere; ACM validation still needs DNS control.
GitHub Actions OIDC	Passwordless CI/CD access to ECR/ECS/Terraform	Do not add long-lived AWS access keys to GitHub secrets.

Adopt after the core system is stable

Service	When it helps	Why it is deferred
CloudFront	Authenticated media-read delivery or public marketing assets	Evidence is intentionally private and has no download API today.
AWS MediaConvert	Large/video evidence transcoding	Add only after product requirements need video derivatives.
AWS Device Farm	Pre-release physical-device matrix	Useful before store submission, not required for the first native vertical slice.
AWS X-Ray / OpenTelemetry	Cross-service tracing	Add once ECS baseline metrics and Sentry are being actively reviewed.
Amazon SES	Transactional email	Twilio already owns OTP delivery; email is not part of today’s flow.
EventBridge	Integration events to external systems	PostgreSQL outboxes + BullMQ already solve internal durable delivery.
Cognito	A deliberate future identity migration	Do not run two OTP/JWT authorities at the same time.
API Gateway, Lambda, AppSync, DynamoDB	A future architecture rewrite	They do not simplify the current Socket.IO, PostGIS, or worker architecture.
SNS/Pinpoint	Future push or campaign consolidation	Existing backend already sends FCM and persists device tokens.

Delivery Phases

Phase 0: Security and Repository Baseline

Objective: Protect credentials and establish the native/AWS boundary before application code or cloud resources exist.

1. Confirm the source fork and protect root-level environment, Terraform state, mobile signing files, and Firebase configuration from Git.
2. Record current API routes, DB lifecycle enums, S3 presigned POST requirements, and web design tokens.
3. Establish the rule that mobile apps use API projections only, never direct database/AWS access.
4. Capture the AWS account owner, deployment region, environment names, domain/DNS owner, cost ceiling, Android application ID, and Apple bundle/team IDs outside source control.

Exit criteria: No credentials are committed; all stakeholders agree on API-as-control-plane and ECS/Fargate as the target runtime.

Phase 1: Contract Hardening

Objective: Make one mobile-safe, versioned contract from the API's actual Zod routes and PostgreSQL lifecycle.

1. Maintain `packages/api-contracts/networkpeer-mobile-contract.yaml` with API envelope, models, lifecycle enums, endpoint inputs, and secure S3 sequence.
2. Promote route schemas into generated OpenAPI/JSON Schema and validate the contract in backend CI.
3. Resolve current type drift before code generation: payment reversal enum, evidence `VERIFIED`, and minimal work-status/submit responses.
4. Add contract tests for response casing, decimal-string cursors, GeoJSON coordinate order, and error semantics.

Exit criteria: Kotlin and Swift clients can be generated or validated against a versioned API contract; a route change cannot silently break a native app.

Phase 2: AWS Landing Zone and Runtime Infrastructure

Objective: Provision a safe, reproducible non-production AWS environment without static credentials in source control.

1. Set up identity roles, CloudTrail, budget alerts, encrypted Terraform state, GitHub OIDC, tags, and least-privilege policies.
2. Create VPC, two-AZ subnet layout, security groups, ECR, ECS cluster, ALB/ACM, private RDS/PostGIS, TLS ElastiCache, and private evidence S3.
3. Put all runtime secrets in Secrets Manager. Create a one-shot ECS migration/provisioning task that runs existing migrations and DB-role provisioning before serving traffic.
4. Add ALB dependency health checks for `/api/v1/health`, ECS task alarms, RDS backup/restore policy, S3 lifecycle policy, and WAF baseline rules.

Exit criteria: A staging API and worker run from immutable ECR image tags; all database roles, S3 checks, and health checks pass using production-like TLS.

Phase 3: Shared Native Foundation and Design System

Objective: Build Android and iOS foundations that visually match the web product and enforce native security expectations.

1. Build Compose and SwiftUI token systems from the web's indigo/teal, slate, radius, surface, chip, card, status, loading, error, and accessibility patterns.
2. Implement Keychain/Keystore token storage, automatic one-time refresh/retry, logout cleanup, API error mapping, role guards, and deep-link-safe navigation.
3. Add local persistence for session and durable sync cursor; queue only retry-safe local operations.
4. Set up native configuration files that contain public API/Stripe keys only. Keep secrets server-side.

Exit criteria: Both apps compile and can perform OTP login, preserve secure session state, handle token rotation, and render the same core visual language as the web app.

Phase 4: Android Kotlin Application

Objective: Deliver the Android client-side vertical slice in Jetpack Compose.

1. Implement client login, job list/detail/create, Stripe PaymentSheet funding, wallet, notifications, and approval.
2. Implement worker native location permission/update, privacy-safe nearby discovery, atomic acceptance, task lifecycle, camera/document selection, SHA-256, presigned S3 POST, confirmation, and submission.
3. Add FCM token registration, data-only account-validated notification hints, notification deep links, Socket.IO recovery, and durable user-scoped worker sync cursor/cache.
4. Test on small and large Android screens, offline/poor network paths, dark mode, TalkBack, and API level support.

Exit criteria: An Android device can complete the same client-to-worker-to-evidence-to-approval live flow as the web demonstration.

Phase 5: iOS Swift Application

Objective: Deliver parity in a native SwiftUI application without translating Android UI literally.

1. Implement the same client and worker vertical slices using SwiftUI navigation, Keychain, URLSession, Core Location, Photos/camera, SHA-256, and native Stripe PaymentSheet.
2. Add APNs/FCM data-only registration, account-validated notification hints/deep links, Socket.IO recovery, and SwiftData sync persistence.
3. Test Dynamic Type, VoiceOver, iPhone compact layouts, iPad layouts where supported, and background/foreground transitions.

Exit criteria: An iPhone can complete the same authoritative lifecycle flow with equivalent navigation, feedback, and security controls.

Phase 6: Cross-Client Realtime, Offline, and Quality

Objective: Make web, Android, and iOS consistent under retries, reconnects, and concurrent activity.

1. Persist string sync cursors, reconcile through `/sync` or `/worker/sync`, and treat Socket.IO as best-effort notification only.

2. Add idempotency keys to all create/fund/approve/evidence requests and cover interrupted upload/retry behavior.
3. Test privacy boundaries: no client identity/address/location pre-assignment; no direct S3 reads; no wallet calculation in clients.
4. Expand API contract/E2E tests to cover each client workflow.

Exit criteria: Cross-device tests show one authoritative result for acceptance, payment settlement, evidence confirmation, and ledger balances.

Phase 7: Release Engineering and Stores

Objective: Make the apps safely deployable through test tracks and store review.

1. Run the committed Android wrapper-based unit-test workflow; add device/UI coverage and an iOS build/test runner once macOS/Xcode signing configuration is available.
2. Sign Android with Play App Signing and iOS with App Store Connect certificates/profiles stored in a secure CI provider.
3. Complete privacy manifests/labels, support URLs, app store metadata, screenshots, crash reporting, and beta distribution through Play Internal Testing/TestFlight.
4. Run Device Farm or a documented real-device matrix before public release.

Exit criteria: Signed release candidates are distributed to internal testers and pass the end-to-end acceptance checklist.

Phase 8: Production Operations

Objective: Operate the AWS deployment predictably after launch.

1. Review CloudWatch/Sentry alarms, Stripe webhook failures, queue/outbox lag, database performance, S3 verification failures, and mobile crash reports.
2. Exercise RDS restore, service rollback, secret rotation, and compromised-token procedures.
3. Review AWS cost, autoscaling, NAT data charges, RDS capacity, and object lifecycle after real usage patterns exist.

Exit criteria: The team can deploy, observe, roll back, and restore the system without relying on an individual developer’s local machine.

First AWS Deployment Inputs Required

Do not paste AWS access keys into this repository or into chat. Before Phase 2 provisioning, provide these non-secret decisions through the team's approved channel:

1. AWS account/environment naming and primary region.
2. Staging versus production scope and monthly cost ceiling.
3. API hostname and DNS/Route 53 ownership; TLS certificate/domain validation owner.
4. Vercel production URL(s) for exact `CORS_ORIGINS`.
5. Existing RDS/Redis/S3 resources to reuse versus new resources to create.
6. Android package ID, iOS bundle ID, Apple Developer Team ID, and Firebase project ownership.
7. Stripe publishable key for mobile configuration. It is public but must still be supplied per environment; Stripe secret/webhook keys remain in Secrets Manager only.

Native S3 Evidence Sequence

1. Worker captures/selects a file locally and calculates the exact SHA-256 digest.
2. Native app calls `POST /work/upload-url` with the job/subtask IDs, MIME type, byte count, capture time, checksum, and a new idempotency key.
3. API reserves metadata in PostgreSQL and returns `upload.url` and opaque form `upload.fields`.
4. Native app performs an S3 multipart **POST** with every provided field and the file. It does not construct its own S3 key or send AWS credentials.
5. Native app calls `POST /work/evidence` with only `media_id`.
6. API reads the S3 object and accepts it only if checksum, content type, exact length, ETag, non-null version ID, and object state match the reservation.
7. Native app calls `POST /work/submit` only after all required evidence records are `UPLOADED`.

Implemented Delivery Artifacts

1. Root secret/state/signing-file ignore rules and public-only native configuration.
2. Versioned mobile contract covering API envelopes, lifecycle actions, evidence review, device registration/deregistration, and secure S3 flow.

3. Android and iOS client/worker workflows with one-time refresh protection, cursor reconciliation, data-only account-validated push hints, Stripe adapters, evidence capture/upload, and wallet/inbox projections.
4. AWS Terraform target, OIDC workflows, immutable API/worker/migrator image release, parked generic Terraform, migration gating, and ECS/ALB verification.
5. Canonical delivery/runbook documentation and an editable DOCX under `docs/`.